

Bài thực hành số 3: Xây dựng và triển khai một ứng dụng phi tập trung (DApp)

1. Mục tiêu

Bài thực hành nhằm giúp sinh viên nắm vững các kiến thức và kỹ năng cơ bản trong việc xây dựng và triển khai một ứng dụng phi tập trung. Sau khi hoàn thành, sinh viên có thể:

- Hiểu quy trình phát triển DApp hoàn chỉnh, từ viết smart contract đến triển khai giao diện người dùng..
- Phát triển ứng dụng front-end phi tập trung bằng PySide (Python GUI) để tương tác trực tiếp với smart contract.
- Kết nối giao diện với blockchain, thực hiện các thao tác như tạo tài khoản, gửi giao dịch, và hiển thị dữ liệu phi tập trung.
- Hiểu được giá trị của tính minh bạch và bảo mật trong các ứng dụng phi tập trung so với mô hình tập trung.

2. Thời gian làm bài

Dự kiến: 3 giờ.

3. Cơ sở lý thuyết và bài tập tại lớp

A. Giới thiệu và Chuẩn bị Môi trường

DApp (Decentralized Application) là ứng dụng được xây dựng trên nền tảng blockchain, trong đó:

- Logic nghiệp vụ được viết dưới dạng smart contract, chạy trên blockchain.
- Dữ liệu được lưu trữ phi tập trung, không có máy chủ trung gian.
- Người dùng tương tác với blockchain qua giao diện (front-end) thông qua thư viện như web3.py.

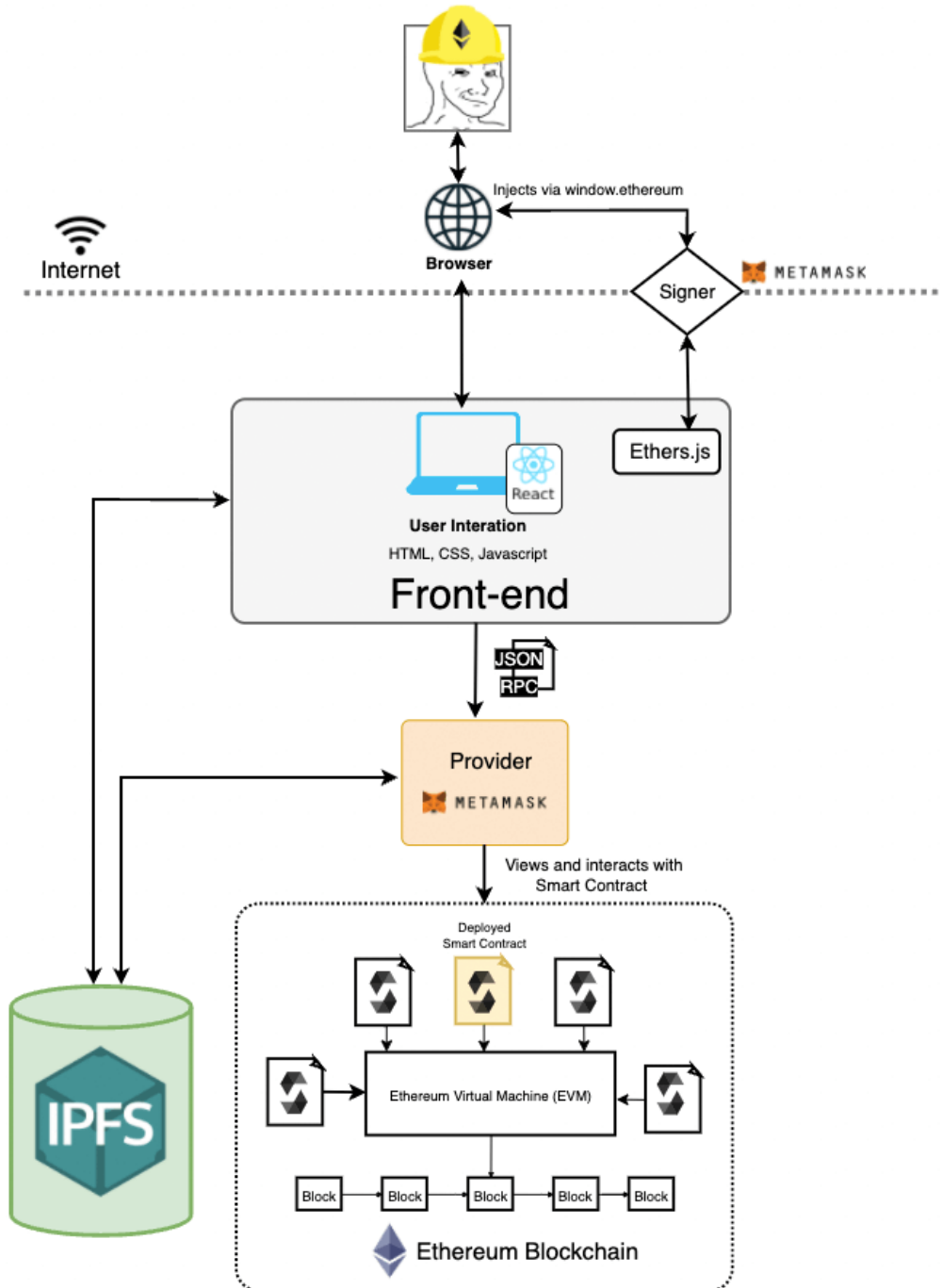
DApp hoạt động theo mô hình số 1. Ưu điểm của ứng dụng:

- Minh bạch, không thể sửa đổi kết quả.
- Không cần máy chủ trung gian, giảm rủi ro tấn công tập trung.
- Dữ liệu và trạng thái giao dịch được xác thực bởi mạng lưới blockchain.

Môi trường chuẩn bị:

- Python – Ngôn ngữ chính để phát triển
- Vyper – Ngôn ngữ viết smart contract
- Ape framework – Framework quản lý dự án Web3
- Geth – Mạng blockchain cục bộ

- PySide – Xây dựng giao diện GUI cho DApp
- Web3.py – Giao tiếp với blockchain qua Python.



Hình 1. Mô hình hoạt động của DApp

i. Thiết lập môi trường làm việc

Bước 1. Tạo môi trường ảo

Bước 2. Cài đặt Ape Framework và các plugin cần thiết (Nếu bài lab 2 chưa thực hiện)

Bước 3. Cài đặt Vyper (ngôn ngữ smart contract) (Nếu bài lab 1 chưa thực hiện)

Bước 4. Cài đặt Geth (Ethereum client cục bộ) (Nếu bài lab 2 chưa thực hiện)

Bước 5. Tạo dự án Ape mới, đặt tên là: voting_dapp. Sau đó tạo các file như hình số 2, khi thực hiện xong các em chụp kết quả của cấu trúc dự án để đưa vào báo cáo.

```
voting_dapp/  
├── contracts/  
│   └── voting.vy  
├── scripts/  
│   ├── deploy.py  
│   ├── vote.py  
│   └── result.py  
├── tests/  
│   └── test_voting.py  
└── ape-config.yaml
```

Hình 2. Cấu trúc dự án

Bước 6. Cài đặt PySide6 cho GUI (Phần front-end). Sử dụng câu lệnh: pip install PySide6

Bước 7. Kiểm tra và xác nhận môi trường. Các em thực hiện các câu lệnh sau:

```
ape --version  
vyper --version  
python -m PySide6 --version
```

ii. Kiến trúc hệ thống DApp

Trong bài thực hành này, DApp được thiết kế gồm ba tầng:

- Smart Contract (Tầng Blockchain):
 - Viết bằng Vyper, triển khai qua Ape Framework.
 - Đảm bảo tính toàn vẹn của dữ liệu bỏ phiếu.
- Backend Scripts (Tầng Giao tiếp):
 - Sử dụng Web3.py để gọi các hàm của smart contract.
 - Thực hiện việc gửi giao dịch, kiểm tra kết quả.
- Frontend GUI (Tầng Người dùng):
 - Viết bằng PySide6, cung cấp giao diện thân thiện.
 - Cho phép người dùng bỏ phiếu và xem kết quả.

B. Xây dựng Smart Contract

Miêu tả bài toán: Hệ thống bỏ phiếu phi tập trung (Voting DApp) được xây dựng nhằm minh họa cách áp dụng công nghệ blockchain trong việc tạo ra một nền tảng bầu chọn minh bạch, an toàn và không thể bị can thiệp. Mục tiêu của hệ thống là cung cấp một cơ chế bỏ phiếu trong đó mọi hành động — từ việc cấp quyền bầu cử, bỏ phiếu, ủy quyền phiếu cho người khác, đến việc công bố kết quả — đều được ghi nhận và xác thực trực tiếp trên blockchain, loại bỏ

hoàn toàn nguy cơ gian lận hoặc thao túng kết quả. Thông qua dự án này, sinh viên có thể hiểu rõ quy trình phát triển một ứng dụng phi tập trung (DApp) thực tế, từ việc viết smart contract bằng ngôn ngữ Vyper, triển khai và kiểm thử bằng Ape Framework, cho đến việc tích hợp với giao diện người dùng (GUI) giúp người dùng cuối có thể dễ dàng tham gia bỏ phiếu. Mục tiêu cuối cùng là giúp sinh viên nắm bắt toàn diện quy trình thiết kế, triển khai, và vận hành một ứng dụng blockchain mang tính ứng dụng cao trong các lĩnh vực như quản trị tổ chức, bầu cử nội bộ hoặc hệ thống biểu quyết trong các tổ chức tự trị phi tập trung (DAO).

Đặc tả chức năng: Hệ thống Voting DApp bao gồm các thành phần và chức năng chính được triển khai trên blockchain Ethereum nhằm đảm bảo tính toàn vẹn và bảo mật của quy trình bầu chọn. Mỗi người dùng (voter) là một địa chỉ ví duy nhất có thể được cấp quyền tham gia bỏ phiếu hoặc ủy quyền phiếu của mình cho người khác thông qua cơ chế delegation. Tất cả các đề xuất (proposals) được khởi tạo bởi người triển khai hợp đồng (chairperson) và lưu trữ vĩnh viễn trên blockchain. Khi người dùng thực hiện hành động bỏ phiếu, hệ thống sẽ xác minh quyền bầu chọn, đánh dấu người đó đã bỏ phiếu, và tăng tổng số phiếu cho đề xuất tương ứng. Ngoài ra, hợp đồng thông minh còn cho phép ủy quyền phiếu bầu, nghĩa là một người có thể chuyển quyền bỏ phiếu của mình cho một người khác nếu chưa bỏ phiếu. Sau khi quá trình bỏ phiếu hoàn tất, hệ thống có thể xác định đề xuất chiến thắng thông qua hàm `winningProposal()` và hiển thị kết quả bằng hàm `winnerName()`. Nhờ việc toàn bộ logic bầu chọn được thực thi trên blockchain, kết quả của Voting DApp mang tính bất biến, minh bạch và có thể kiểm chứng công khai, đảm bảo niềm tin tuyệt đối cho người tham gia.

Thành phần	Mô tả
Người bỏ phiếu (Voter)	Có quyền bỏ phiếu hoặc ủy quyền phiếu cho người khác
Đề xuất (Proposal)	Danh sách các lựa chọn (ví dụ: ứng cử viên hoặc phương án)
Ủy quyền (Delegation)	Voter có thể ủy quyền phiếu cho voter khác
Bỏ phiếu	Mỗi người chỉ được bỏ một phiếu duy nhất
Tính kết quả	Xác định đề xuất thắng cuộc dựa trên số phiếu

Hướng dẫn thực hiện các bước:

- Viết smart contract bằng Vyper: định nghĩa cấu trúc đề xuất, người bỏ phiếu, và cơ chế ủy quyền (delegation).
- Thêm logic kiểm tra hợp lệ và ghi nhận sự kiện bỏ phiếu.
- Viết unit tests để xác minh các hàm chính (`vote`, `delegate`, `winningProposal`).
- Biên dịch và chạy test

C. Triển khai và Tương tác Smart Contract

i. Khai báo cấu trúc dữ liệu

```
struct Voter:
    weight: uint256
    voted: bool
    delegate: address
    vote: uint256

struct Proposal:
    name: String[100]
    voteCount: uint256
```

ii. Biến trạng thái chính

```
voters: public(HashMap[address, Voter])
proposals: public(HashMap[uint256, Proposal])
voterCount: public(uint256)
chairperson: public(address)
amountProposals: public(uint256)

MAX_NUM_PROPOSALS: constant(uint256) = 3
```

iii. Hàm khởi tạo

Gán chairperson là người tạo contract.

```
@deploy
def __init__():
    self.chairperson = msg.sender
```

Khởi tạo danh sách đề xuất.

```
@external
def addProposal(_proposalName: String[100]):
    assert msg.sender == self.chairperson
    i: uint256 = self.amountProposals
    self.proposals[i].name = _proposalName
    self.proposals[i].voteCount = 0
    self.amountProposals += 1
```

iv. Các hàm chức năng

Hàm	Chức năng
giveRightToVote(voter: address)	Cho phép một địa chỉ có quyền bỏ phiếu
delegate(to: address)	Ủy quyền phiếu bầu
vote(proposal: uint256)	Bỏ phiếu cho một đề xuất
winningProposal() -> uint256	Xác định đề xuất có nhiều phiếu nhất
winnerName() -> String[32]	Trả về tên của đề xuất thắng cuộc

Chi tiết các hàm

```
@external
def giveRightToVote(voter: address, _weight: uint256):
    assert msg.sender == self.chairperson
    assert not self.voters[voter].voted
    assert self.voters[voter].weight == 0
    self.voters[voter].weight = _weight
    self.voterCount += 1
```

```
@external
def delegate(to: address):
    assert not self.voters[msg.sender].voted
    assert to != msg.sender
    assert to != empty(address)

    self.voters[msg.sender].voted = True
    self.voters[msg.sender].delegate = to

    self._forwardWeight(msg.sender)
```

```
@external
def vote(proposal: uint256):
    assert not self.voters[msg.sender].voted
    assert proposal < self.amountProposals

    self.voters[msg.sender].vote = proposal
    self.voters[msg.sender].voted = True

    self.proposals[proposal].voteCount += self.voters[msg.sender].weight
    self.voters[msg.sender].weight = 0
```

```

@view
@internal
def _winningProposal() -> uint256:
    winning_vote_count: uint256 = 0
    winning_proposal: uint256 = 0
    for i in range(MAX_NUM_PROPOSALS):
        if self.proposals[i].voteCount > winning_vote_count:
            winning_vote_count = self.proposals[i].voteCount
            winning_proposal = i
    return winning_proposal

@view
@external
def winningProposal() -> uint256:
    return self._winningProposal()

@view
@external
def winnerName() -> String[100]:
    return self.proposals[self._winningProposal()].name

```

v. Miêu tả hoạt động

- Triển khai smart contract:
 - Chủ tịch (chairperson) triển khai contract lên mạng thử nghiệm Geth.
 - Khởi tạo danh sách đề xuất.
- Cấp quyền bỏ phiếu:
 - Chairperson gọi hàm giveRightToVote() cho các địa chỉ hợp lệ.
- Người dùng bỏ phiếu hoặc ủy quyền:
 - Người dùng gọi vote() hoặc delegate() thông qua GUI.
- Tổng hợp kết quả:
 - Hàm winningProposal() và winnerName() được gọi để xác định kết quả.

vi. Biên dịch và kiểm thử hợp đồng

Trong thư mục dự án, thực hiện lệnh sau: `ape compile`. Sau khi compile thành công thì thực hiện triển khai dự án trên Geth như bài thực hành số 2. Chụp ghi lại địa chỉ của smart contract khi deploy thành công.

```

import pytest

@pytest.fixture
def deployer(accounts):
    return accounts[0]

@pytest.fixture
def contract(deployer, project):
    return deployer.deploy(project.VotingApp)

@pytest.fixture
def delegate_contract(deployer, project):
    return deployer.deploy(project.DelegateVotingApp)

```

Hình 3. file kiểm thử hoạt động của smart contract

Viết 1 chương trình kiểm thử hoạt động smart contract, tạo 1 file tên confest.py như hình số 3. Sau đó thực hiện lệnh ape test.

Sau đó tạo 2 file test_voting_app.py trong thư mục test để kiểm thử các function của smart contract.

```
import pytest
from ape.exceptions import ContractLogicError

def test_chairperson(contract, deployer):
    chairperson = contract.chairperson()
    assert chairperson == deployer

def test_addProposal(contract, deployer):
    assert contract.amountProposals() == 0
    assert contract.proposals(0).name == ""
    contract.addProposal("beach", sender=deployer)
    assert contract.amountProposals() == 1
    assert contract.proposals(0).name == "beach"

def test_addProposal_fail(contract, accounts):
    with pytest.raises(ContractLogicError):
        contract.addProposal("beach", sender=accounts[1])

def test_giveRightToVote(contract, deployer, accounts):
    user = accounts[1]
    assert contract.voterCount() == 0
    assert contract.voters(user).weight == 0
    contract.giveRightToVote(user, 1, sender=deployer)
    assert contract.voterCount() == 1
    assert contract.voters(user).weight == 1

    power_user = accounts[2]
    assert contract.voters(power_user).weight == 0
    contract.giveRightToVote(power_user, 9, sender=deployer)
    assert contract.voterCount() == 2
    assert contract.voters(power_user).weight == 9

def test_giveRightToVote_fail(contract, deployer, accounts):
    user = accounts[1]
    with pytest.raises(ContractLogicError):
        contract.giveRightToVote(user, 1, sender=accounts[1])

def test_vote_fail(contract, deployer, accounts):
    contract.addProposal("beach", sender=deployer)
    contract.addProposal("mountain", sender=deployer)
    user = accounts[1]
    contract.giveRightToVote(user, 1, sender=deployer)

    contract.vote(0, sender=user)
    with pytest.raises(ContractLogicError):
        contract.vote(0, sender=user)
```

```

def test_vote(contract, deployer, accounts):
    contract.addProposal("beach", sender=deployer)
    contract.addProposal("mountain", sender=deployer)
    user = accounts[1]
    user2 = accounts[2]
    contract.giveRightToVote(user, 1, sender=deployer)
    contract.giveRightToVote(user2, 1, sender=deployer)

    assert contract.voters(user).weight == 1
    assert contract.voters(user).voted == False
    assert contract.voters(user).vote == 0
    assert contract.proposals(0).voteCount == 0
    contract.vote(0, sender=user)
    assert contract.proposals(0).voteCount == 1
    assert contract.voters(user).weight == 0
    assert contract.voters(user).voted == True
    assert contract.voters(user).vote == 0

    assert contract.voters(user2).weight == 1
    assert contract.voters(user2).voted == False
    assert contract.voters(user2).vote == 0
    assert contract.proposals(1).voteCount == 0
    contract.vote(1, sender=user2)
    assert contract.proposals(1).voteCount == 1
    assert contract.voters(user2).weight == 0
    assert contract.voters(user2).voted == True
    assert contract.voters(user2).vote == 1

def test_winnerName(contract, deployer, accounts):
    contract.addProposal("beach", sender=deployer)
    contract.addProposal("mountain", sender=deployer)
    user1 = accounts[1]
    user2 = accounts[2]
    user3 = accounts[3]
    contract.giveRightToVote(user1, 1, sender=deployer)
    contract.giveRightToVote(user2, 1, sender=deployer)
    contract.giveRightToVote(user3, 1, sender=deployer)

    contract.vote(0, sender=user1)
    contract.vote(1, sender=user2)
    contract.vote(1, sender=user3)

    assert contract.proposals(0).voteCount == 1
    assert contract.proposals(1).voteCount == 2
    assert contract.winnerName() == "mountain"

```

Sau khi viết xong đoạn chương trình trên, các em biên dịch và chạy chương trình test_voting_app.py

Khi thực hiện test các chức năng của hàm xong, các em hãy chụp hình kết quả chạy và giải thích chi tiết từng function trong file test_voting_app.py đưa vào báo cáo, nộp bài tập tại lớp

Thực hiện deploy smart contract lên blockchain Geth, các em tạo file deploy.py trong thư mục scripts, có nội dung như hình số 4. Sau đó chạy lệnh `ape run deploy --network ethereum:local:http://127.0.0.1:8545`.

Các em chụp lại kết quả khi deploy thành công để đưa vào báo cáo nộp bài tập tại lớp.

```

from ape import accounts, project
import os

def main():
    password = "CHAIRPERSON_PASSWORD"
    chairperson = accounts.load("chairperson")
    chairperson.set_autosign(True, passphrase=password)
    contract = project.VotingApp.deploy(sender=chairperson)
    chairperson = contract.chairperson()
    print(f"The chairperson account is {chairperson}")

```

Hình 4. File Deploy.py thực hiện deploy smart contract lên blockchain Geth

Các em tạo file give_right_to_vote.py trong thư mục scripts có nội dung như hình 5, sau đó thực hiện run 3 lần (có thể nhiều hơn) thực hiện việc bầu chọn các lần khác nhau, các em chụp hình kết quả chạy, và thực hiện giải thích kết quả chạy.

```

from ape import accounts, project
import os

def main():
    password = "CHAIRPERSON_PASSWORD"
    address = "VOTING_APP_ADDRESS"
    voter_1 = "VOTER_1_ADDRESS"
    voter_2 = "VOTER_2_ADDRESS"
    voter_3 = "VOTER_3_ADDRESS"
    chairperson = accounts.load("chairperson")
    chairperson.set_autosign(True, passphrase=password)
    contract = project.VotingApp.at(address)
    contract.giveRightToVote(voter_1, 1, sender=chairperson)
    contract.giveRightToVote(voter_2, 1, sender=chairperson)
    contract.giveRightToVote(voter_3, 1, sender=chairperson)

```

Hình 5. File thực hiện việc bỏ phiếu

Tạo 1 file vote.py trong thư mục scripts, có nội dung như hình 6. Đoạn chương trình này thực hiện bỏ phiếu bởi các tài khoản voter chứ không phải là từ tài khoản chairperson.

```

from ape import accounts, project
import os

def main():
    password = "VOTER_PASSWORD"
    address = "VOTING_APP_ADDRESS"
    voter_account = "VOTER_ACCOUNT"
    proposal = "PROPOSAL"
    voter = accounts.load(voter_account)
    voter.set_autosign(True, passphrase=password)
    contract = project.VotingApp.at(address)
    contract.vote(proposal, sender=voter)

```

Hình 6. Đoạn chương trình thực hiện bỏ phiếu bởi tài khoản voter

D. Phát triển Ứng dụng Front-End (GUI DApp)

Tạo 3 tài khoản người dùng: voter1, voter2, voter3, và chairperson.

Sử dụng câu lệnh sau để tạo voter1, tương tự như lúc tạo tài khoản is355 trong bài thực hành số 2 : `ape accounts generate voter1`

Làm tương tự để tạo tài khoản voter2, voter3, và chairperson. Sau khi thực hiện tạo tài khoản xong, hãy chụp hình cho biết kết quả tạo thành công.

Thực hiện chạy blockchain Geth, kiểm tra thông tin các tài khoản vừa tạo có trên mạng cục bộ chưa? Sử dụng câu lệnh: `geth attach /tmp/geth.ipc` (Đối với hệ điều hành window, khi các em start Geth để ý đường dẫn file `geth.ipc` có thể khác, các em lấy đường dẫn file `geth.ipc` trong cửa sổ console start mạng Geth).

Các em chạy thử các câu lệnh sau để kiểm tra các tài khoản cụ thể và chụp kết quả chạy từng câu lệnh:

```
web3.fromWei(eth.getBalance(eth.accounts[0]))
```

```
eth.sendTransaction({from: eth.accounts[0], to: "địa chỉ người nhận", value: web3.toWei(10, "ether")}).
```

Tiếp theo các em thực hiện viết giao diện để tương tác với smart contract, các em tạo file `voting_gui_app.py` có nội dung như sau:

```
from ape import accounts, project
import os, sys
from PySide6 import QtWidgets, QtCore

class VotingWidget(QtWidgets.QWidget):
    def __init__(self):
        super().__init__()

        self.label = QtWidgets.QLabel("Voting Blockchain App")

        self.combobox = QtWidgets.QComboBox()
        self.combobox.addItem(["----"])

        self.button = QtWidgets.QPushButton("Vote")

        self.layout = QtWidgets.QVBoxLayout(self)
        self.layout.addWidget(self.label)
        self.layout.addWidget(self.combobox)
        self.layout.addWidget(self.button)

        self.button.clicked.connect(self.button_on_clicked)

    @QtCore.Slot()
    def button_on_clicked(self):
        proposal = self.combobox.currentText()
        index = self.combobox.currentIndex()
        print(f"The selected proposal is {proposal}")
        print(f"The selected index is {index}")

        password = "VOTER_PASSWORD"
        address = "VOTING_APP_ADDRESS"
        voter_account = "VOTER_ACCOUNT"
        voter = accounts.load(voter_account)
        voter.set_autosign(True, passphrase=password)

        contract = project.VotingApp.at(address)
        contract.vote(index-1, sender=voter)
```

```
def main():
    voting_app_address = "VOTING_APP_ADDRESS"
    contract = project.VotingApp.at(voting_app_address)
    amount_proposals = contract.amountProposals()

    app = QtWidgets.QApplication([])

    widget = VotingWidget()
    widget.show()

    for i in range(amount_proposals):
        proposal_name = contract.proposals(i).name
        widget.combobox.addItem(proposal_name)

    sys.exit(app.exec())
```

Sau khi thực hiện xong các em compile và chạy chương trình rồi chụp kết quả để đưa vào báo cáo. Lưu ý: VOTING_APP_ADDRESS, VOTER_PASSWORD, VOTING_APP_ADDRESS, VOTER_ACCOUNT đây là các thông tin các em nên lưu lại trong file .txt để trong project và sử dụng đọc file.

4. Bài tập về nhà

Bài xây dựng ứng dụng phi tập trung quản lý Hồ sơ sức khỏe điện tử (Digital Health Records DApp).

A. Mô tả bài toán

Hệ thống Digital Health Records DApp (DHR-DApp) được thiết kế nhằm minh họa việc ứng dụng công nghệ blockchain trong quản lý hồ sơ y tế cá nhân một cách an toàn, minh bạch và phi tập trung.

Trong hệ thống này, bệnh nhân là người sở hữu dữ liệu y tế của chính mình; họ có thể cấp quyền truy cập cho bác sĩ hoặc cơ sở y tế xem, thêm, hoặc cập nhật thông tin điều trị. Tất cả thao tác truy cập và chỉnh sửa đều được ghi lại trên blockchain, đảm bảo tính toàn vẹn và không thể thay đổi của dữ liệu.

Ứng dụng cho phép lưu trữ các thông tin sức khỏe cơ bản như: thông tin cá nhân, lịch sử khám bệnh, chẩn đoán, đơn thuốc, và kết quả xét nghiệm. Thông qua cơ chế smart contract, người dùng có thể kiểm soát hoàn toàn dữ liệu cá nhân và quyền chia sẻ của mình — điều mà các hệ thống tập trung truyền thống không đảm bảo được.

B. Đặc tả chức năng

Hệ thống DHR-DApp bao gồm các chức năng chính sau:

Đối tượng	Chức năng	Mô tả chi tiết
Bệnh nhân (Patient)	- Tạo hồ sơ sức khỏe cá nhân - Cấp quyền truy cập cho bác sĩ - Xem lịch sử truy cập và chỉnh sửa	Bệnh nhân là chủ sở hữu dữ liệu, có thể cho phép hoặc thu hồi quyền truy cập của bác sĩ bất kỳ lúc nào.
Bác sĩ (Doctor)	Xem hồ sơ bệnh nhân được cấp	Mọi hành động cập nhật đều

	quyền - Cập nhật chẩn đoán, đơn thuốc, hoặc kết quả khám	được ghi nhận trên blockchain với địa chỉ ví của bác sĩ.
Cơ sở y tế (Hospital)	- Xác thực danh tính bác sĩ - Quản lý danh sách bác sĩ thuộc hệ thống	Là bên xác nhận quyền hành nghề và đảm bảo tính hợp lệ của bác sĩ trên blockchain.
Smart Contract	- Lưu trữ dữ liệu hồ sơ sức khỏe - Quản lý quyền truy cập (Access Control) - Ghi log các thao tác chỉnh sửa	Mọi thay đổi trong hồ sơ y tế được xác thực và ghi nhận bất biến.

C. Cấu trúc hệ thống

Ứng dụng gồm 3 tầng chính:

- Smart Contract (Blockchain Layer)
 - Viết bằng Vyper, triển khai bằng Ape Framework.
 - Quản lý danh sách bệnh nhân, bác sĩ, và hồ sơ sức khỏe.
 - Đảm bảo mọi thao tác (tạo hồ sơ, truy cập, chỉnh sửa) đều được xác thực và lưu vết.
- Backend Script (Interaction Layer)
 - Viết bằng Python sử dụng Web3.py để gọi hàm của smart contract.
 - Thực hiện việc đăng ký người dùng, cập nhật thông tin, và ghi nhận log giao dịch.
- Frontend GUI (User Layer)
 - Xây dựng bằng PySide6, cung cấp giao diện trực quan.
- Cho phép người dùng:
 - Đăng nhập bằng ví Ethereum (bệnh nhân hoặc bác sĩ)
 - Xem, tạo hoặc chỉnh sửa hồ sơ sức khỏe
 - Theo dõi lịch sử truy cập và cập nhật

D. Yêu cầu giao diện

Giao diện GUI cần được chia thành các tab chức năng như sau:

- Đăng nhập (Login Panel)
 - Người dùng nhập địa chỉ ví và mật khẩu để xác định vai trò (bệnh nhân hoặc bác sĩ).
 - Kết nối tới blockchain qua RPC endpoint (ví dụ: <http://127.0.0.1:8545>).
- Quản lý hồ sơ (Health Records Panel)
 - Hiện thị thông tin hồ sơ sức khỏe của bệnh nhân.
 - Nút “Thêm hồ sơ mới” cho phép nhập thông tin: chẩn đoán, đơn thuốc, ngày khám, ghi chú.
- Cấp quyền truy cập (Access Control Panel)
 - Bệnh nhân nhập địa chỉ ví của bác sĩ và cấp quyền đọc/ghi hồ sơ.
 - Có thể thu hồi quyền bất kỳ lúc nào.
- Lịch sử truy cập (Activity Log Panel)

- Hiện thị danh sách các hành động được ghi lại từ blockchain: ai đã truy cập, khi nào, và thao tác gì.
- Thông báo và xác nhận giao dịch (Transaction Status Panel)
 - Mỗi khi gửi giao dịch lên blockchain, giao diện hiển thị transaction hash, trạng thái pending/confirmed, và thông báo khi hoàn tất.

E. Yêu cầu thực hiện

- Viết smart contract DHR.vy định nghĩa các cấu trúc:
 - PatientRecord, DoctorInfo, AccessPermission
- Cài đặt các hàm chính:
 - registerPatient(), registerDoctor(),
 - grantAccess(address doctor), revokeAccess(address doctor),
 - addHealthRecord(string diagnosis, string prescription, string note),
 - viewHealthRecord(address patient)
- Viết script deploy.py để triển khai hợp đồng lên mạng Geth cục bộ.
- Viết chương trình dhr_gui_app.py để tạo giao diện người dùng với các tab đã mô tả.
- Thực hiện kiểm thử: tạo 2 bệnh nhân và 1 bác sĩ, cấp quyền, ghi hồ sơ mới và xác nhận log giao dịch.